

COMP.CS.510-2025-2026-1 Advanced Web Development: Back End

Group 201

Vilma Pirilä, vilma.pirila@tuni.fi

Elsi Häkkinen, elsi.hakkinen@tuni.fi

Juho Peltonen, juho.peltonen@tuni.fi

<https://course-gitlab.tuni.fi/compcs510-spring2026/201/>

Timetable

Week 1, March 9th to March 15th: Group formation, first meetup, checking project requirements, starting the documentation. Docker Compose setup.

Week 2, March 16th to 22nd: Second meetup. Picking the technologies, learning about NGINX, RabbitMQ and Docker. Randomizing the generator. Loose work division: Vilma in charge of frontend and communication to frontend + graphics, Elsi works in backend server setup and server communication including generator, Juho as the member with most experience handles dockerfiles, does testing and works a full-stack role.

Week 3, March 23rd to March 29th: Work focus on server A and frontend. Group coding session during third meetup on the 26th.

Week 4, March 30th to April 5th: Easter holidays take over most of the week. Independent study. Graphics and server development. Due to incomplete testing, they were not pushed to the main branch yet.

Week 5, April 6th to April 12th: Checking that documentation is up to date. MIDWAY PROJECT CHECK on the 7th. Fourth meetup. Checking the branches + anything the members have worked on but not yet pushed to GitLab. Focus on server B.

Week 6, April 13th to April 19th: Finalising server communication and adding graphics. Testing.

Week 7, April 20th to April 28th: Final week of project, deadline on Tuesday 28th. Final fixes.

Members commit to spending a **minimum of 3 hours** on independent study and development each week, on top of **1-4 hours of weekly meetup**. Actual spent hours should aim to be above this, but fluctuation during the project is allowed, as long as the promised minimum is reached every week.

Architecture

Frontend: React

Backend: Node.js

Considered but not used: Vue.js, TypeScript.

Frontend: Single-Page Application built with React 18. It is served by an nginx web server running inside a Docker container. Its responsibilities are to deliver the compiled JavaScript application to the browser, and to act as a reverse proxy that routes browser WebSocket connections to Server A and REST API calls to Server.

Server A: Server A is a stateless event relay microservice. It is responsible for bridging asynchronous messages arriving from the RabbitMQ message broker to real-time browser clients over the WebSocket protocol. It only routes, without adding any business logic.

Server B: Server B is an event-driven analytics microservice, which processes real-time emoji reactions to detect meaningful moments based on bursts of reactions. RabbitMQ Broker handles its message routing. Server B connects to RabbitMQ, consumes emoji reactions, and detects spikes by maintaining a sliding time window.

Generator: The Generator produces a randomized continuous stream of emoji reaction events and publishes them to a RabbitMQ message exchange. Its purpose is to simulate real-time audience engagement traffic. It operates on a recursive async event loop.

How to try the system:

- Ensure Docker is running on the device
- Open terminal in the project root folder
- Terminal: docker compose up

Server A and –B will fail to connect at first but automatically retry once rabbitmq initializes.

Learnings during project

Elsi: Week 1: what is nginx, general idea of RabbitMQ; Week 3: Learning to use Docker properly

Juho: Using rabbitmq channels and nginx

Vilma: Websocket server, how Docker files work, nginx conf file, rabbitmq